Arquitectura y diseño técnico

HUERTOS MEDICINALES COMUNITARIOS

ALLPANA

Informe de la arquitectura implementada, el modelo de clases y los flujos críticos del prototipo.

Monolito modular · Java 21 · Spring Boot · PostgreSQL

Ariel | Integrante 2

Versión 1.0 | Corte documental: 12 de noviembre de 2026

Rama local: Ariel_Baldeon | Base de Develop: 9a7bd05

Incluye 3 láminas de clases UML, 2 secuencias y despliegue local.

01. Alcance y criterio de lectura

La documentación representa el código local refactorizado, incluidos cambios aún sin commit.

Objetivo

Explicar cómo está construido ALLPANA, qué responsabilidades tiene cada parte y cómo colaboran sus clases. El informe cubre los cinco puntos de la actividad: arquitectura, componentes, clases, secuencias y despliegue.

Método de comprobación

Se contrastaron clases de dominio, servicios, controladores, migraciones, configuración y archivos de despliegue. Los resultados de pruebas se toman de la verificación local registrada; esta entrega documental no vuelve a ejecutar la suite ni altera el negocio.

Qué implementa el prototipo

Gestiona comunidades, miembros, huertos, catálogo de especies, plantas, actividades con evidencias, revisión humana, misiones, puntos, ranking y una bodega básica. La interfaz pública muestra mapa y fichas; las áreas privadas dependen del rol y del recurso autorizado.

Cómo leer los diagramas

Las clases muestran una selección de atributos y operaciones reales. Se omiten getters repetitivos, constructores y campos secundarios. Las asociaciones de dominio son referencias por ID/FK; no significan navegación entre entidades JPA. Los elementos repetidos entre láminas se marcan como referencia.

Estado observado: implementación local. Un diagrama describe su diseño; no certifica que todo requisito o caso límite esté resuelto.

02. Estilo arquitectónico y justificación

Un único proyecto Maven, un JAR de aplicación y una base de datos compartida.

Monolito modular por capas

Los módulos agrupan capacidades del negocio y se dividen en presentación, aplicación, dominio, persistencia y configuración cuando las necesitan. Se despliegan juntos y se comunican mediante llamadas Java síncronas. Thymeleaf y los recursos del navegador se empaquetan en el mismo JAR.

Por qué encaja con el prototipo

Permite coordinar revisión, actividad, recompensas y misión dentro de una transacción de PostgreSQL. Reduce la operación de infraestructura y conserva el estilo de programación existente. La separación por capas delimita cambios sin exigir reescribir el proyecto.

Costo y restricciones

La base y el despliegue son compartidos. Los módulos no son servicios autónomos y existen dependencias entre sus casos de uso. Cambiar una entidad o una migración puede afectar varias capacidades; por ello se conservan contratos de lectura y pruebas de integración.

Precisión del término

No se presenta como arquitectura hexagonal pura: el dominio mantiene anotaciones JPA y los servicios de aplicación usan repositorios concretos. El puerto de archivos permite sustituir ese adaptador, pero no convierte por sí solo todo el sistema en puertos y adaptadores.

03. Componentes y responsabilidades

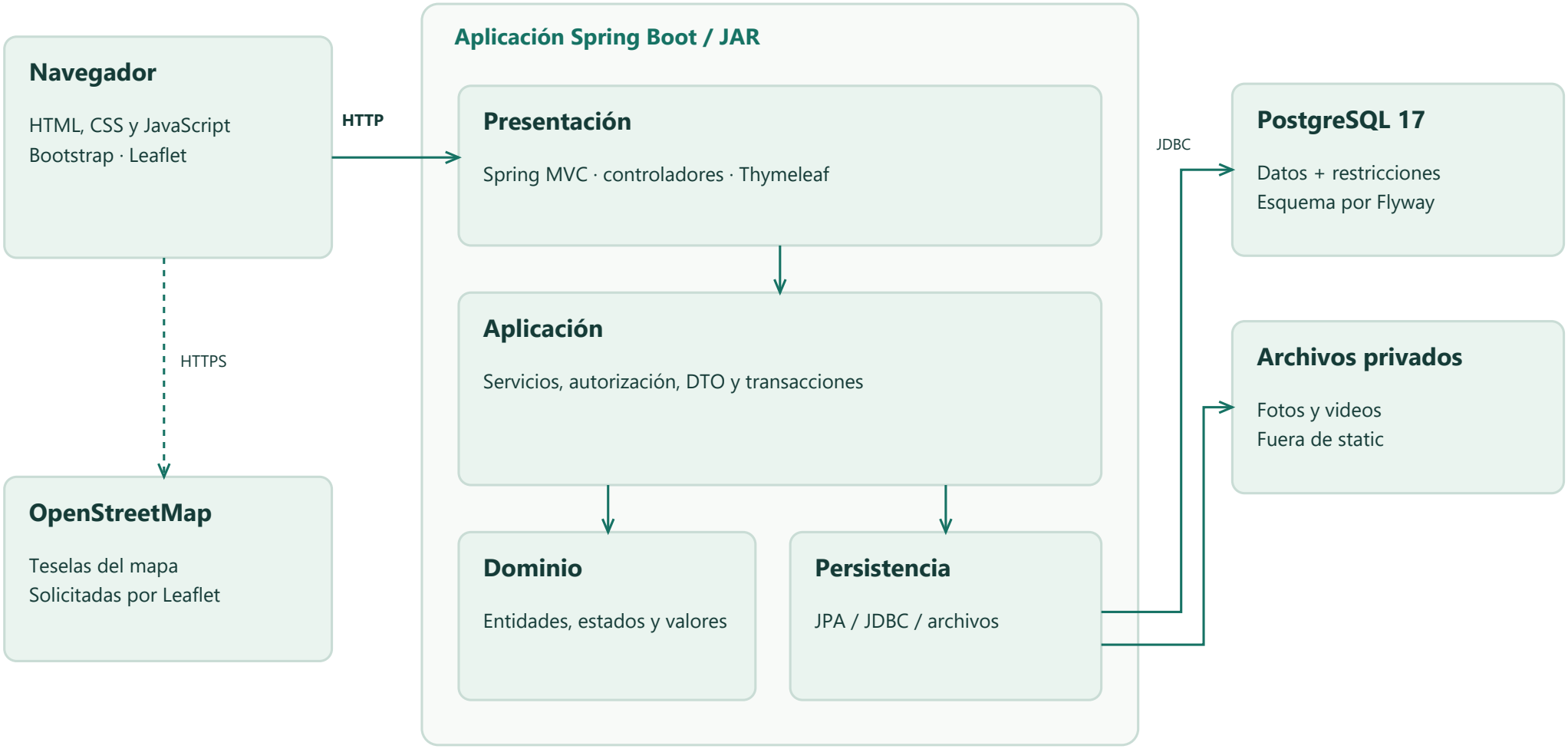
La división funcional coincide con los paquetes productivos bajo ec.edu.epn.huertos.

Módulo	Responsabilidad principal
autenticacion	Registro, usuarios, roles, autorización y recarga de identidad de sesión.
comunidades	Membresías, invitaciones, cupos, solicitudes, asignación de revisores y archivo/restauración.
huertos	Huerto real, espacios, cerramientos, especies compartidas, plantas y lectura del huerto virtual.
actividades / evidencias	Registro y participantes; validación y almacenamiento de archivos; envíos y revisión humana.
misiones / gamificacion	Plantillas y dependencias, estados por huerto, recompensas individuales/comunitarias y ranking.
bodega / publico	Inventario básico por huerto; portada, mapa y fichas para visitantes.
compartido / demo	Fechas, excepciones y errores comunes; carga sintética condicional del perfil demo.

Los módulos colaboran en aplicación. No hay una base de datos separada ni un proceso independiente por módulo.

04. Diagrama de arquitectura

Vista de ejecución y comunicación; todas las capas internas pertenecen al mismo proceso Java.



Configuración conecta Security y propiedades. FFprobe es un proceso local para validar video; no un servicio remoto.

05. Fronteras y organización física

Las reglas se aplican por capa dentro de cada módulo funcional.

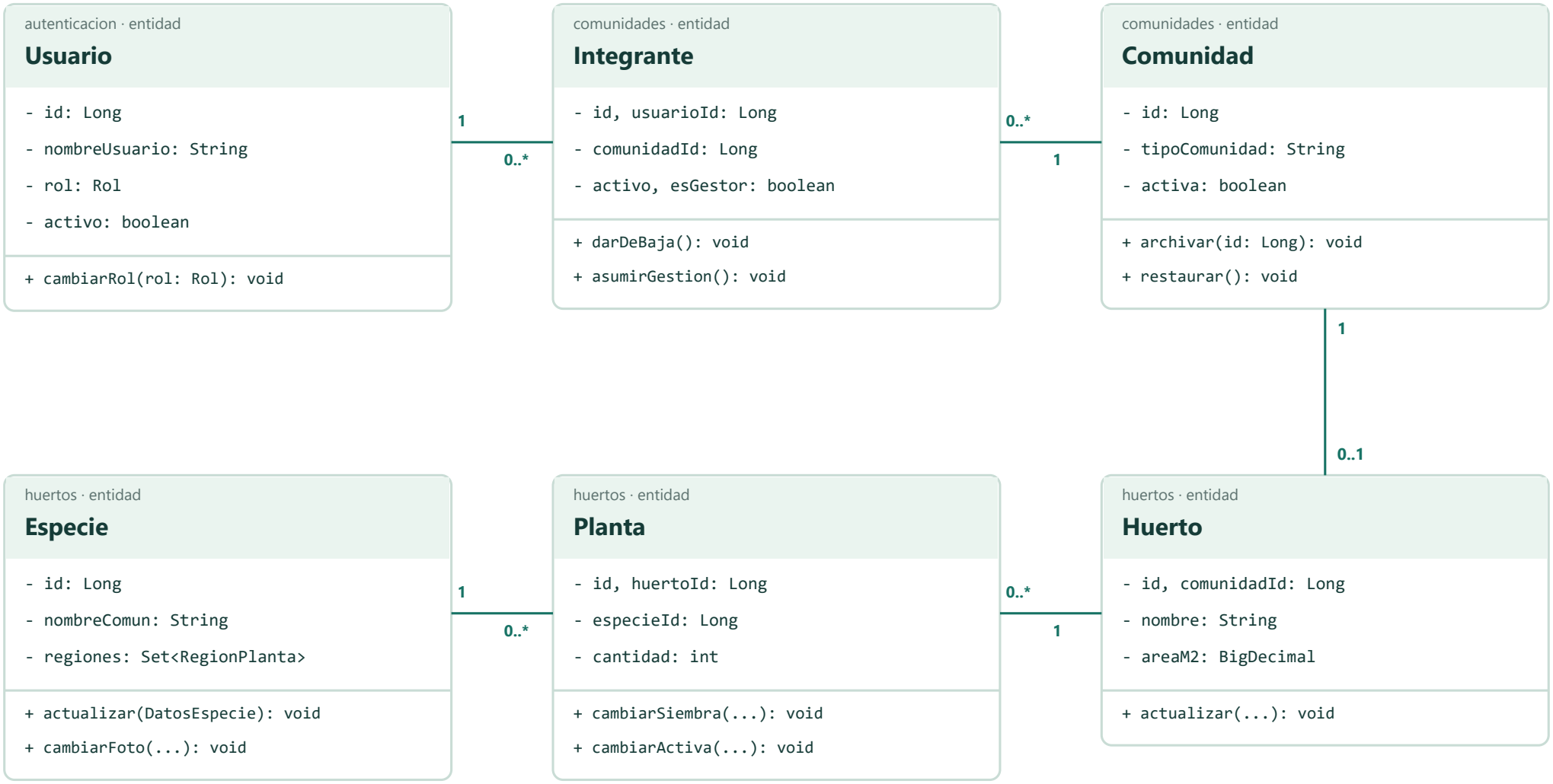
Capa	Contrato y dependencias permitidas
presentacion	Adapta HTTP. Consume servicios y DTO de aplicación. No usa repositorios, SQL ni entidades JPA, tampoco anidadas.
aplicacion	Coordina casos de uso, permisos y transacciones. Usa dominio y persistencia. No depende de controladores ni ejecuta JDBC/EntityManager.
dominio	Contiene entidades, enums, valores y reglas. No depende de las otras capas; conserva las anotaciones JPA existentes.
persistencia	Implementa consultas, joins, bloqueos y archivos. Usa dominio; puede devolver proyecciones de consulta de aplicación.
configuracion	Conecta infraestructura, propiedades y seguridad. No sustituye a los servicios del negocio.

Ubicación: src/main/java/ec/edu/epn/huertos/<modulo>/<capa>/. Las migraciones Java están en db/catalogo y las SQL en src/main/resources/db/migration. Las vistas activas están en frontend/templates y los recursos en frontend/static.

La excepción de proyecciones persistencia → aplicación está documentada. No se declara un grafo acíclico de módulos.

06. Clases: comunidad y huerto

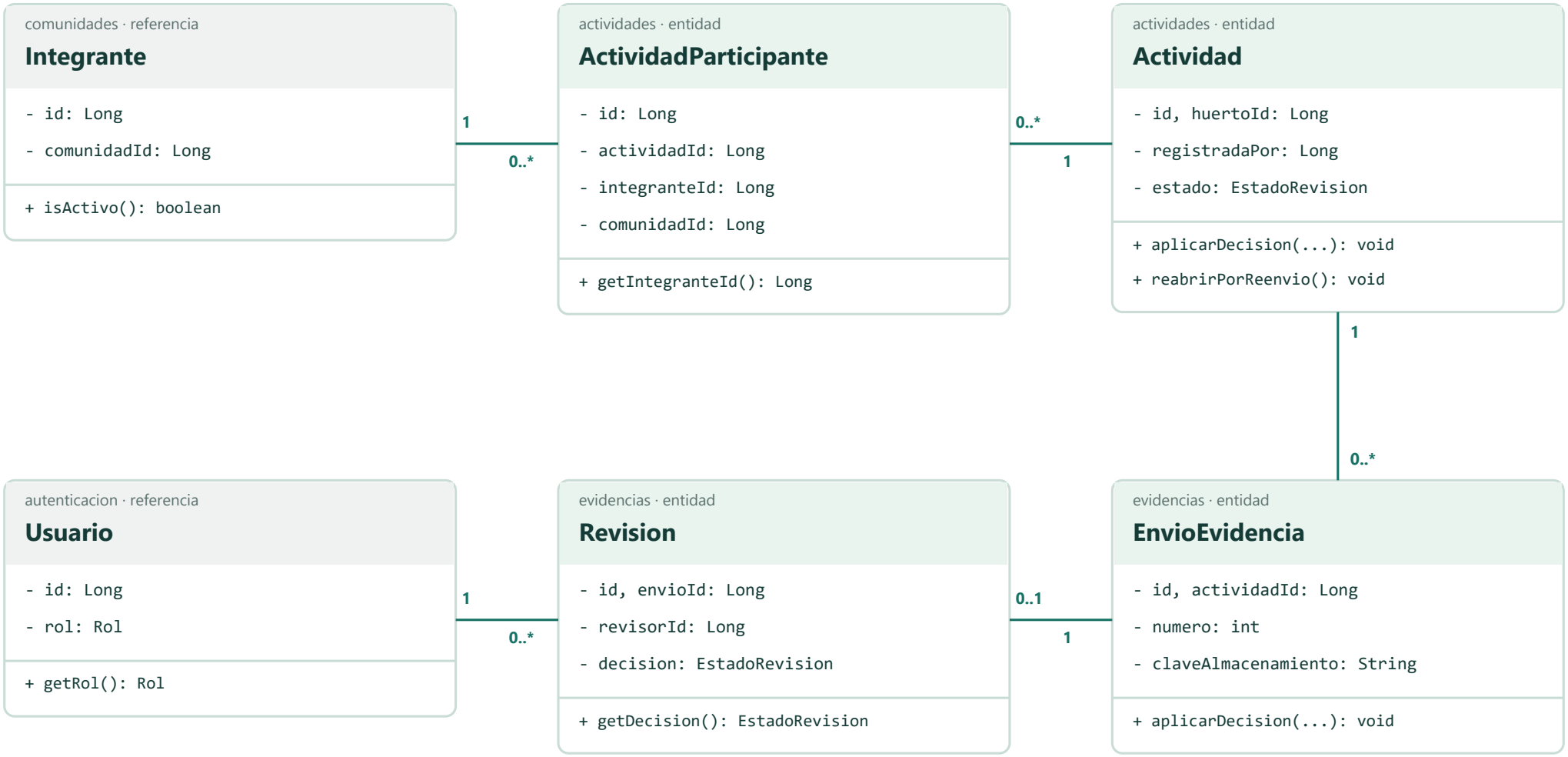
Atributos principales y métodos seleccionados. Asociación sólida = relación referencial por ID/FK.



Usuario puede tener membresías históricas; solo una activa. Planta usa especieId, sin @ManyToOne.

07. Clases: actividades y evidencias

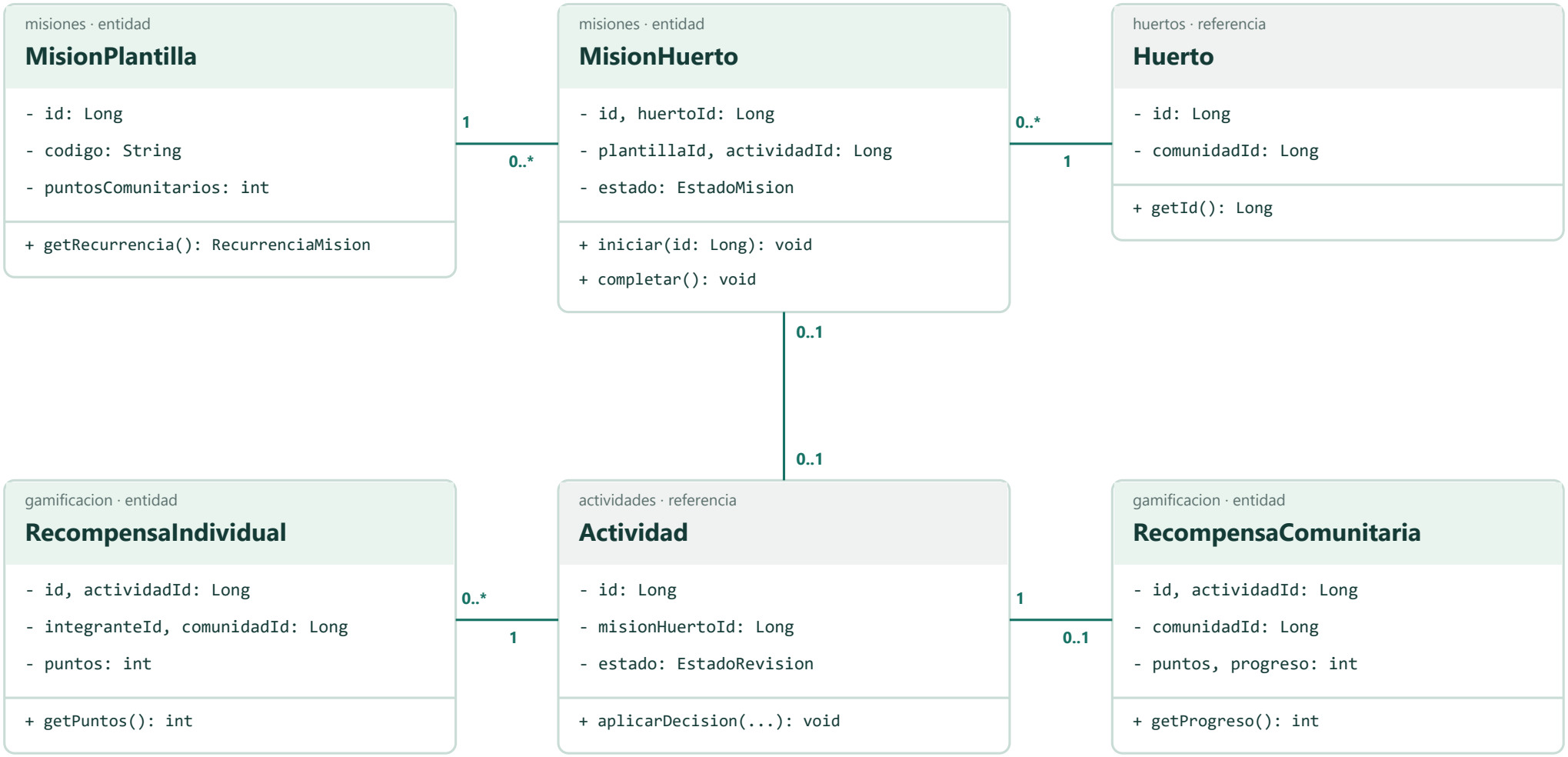
El historial conserva envíos anteriores; cada envío admite una única revisión.



Una alta correcta crea el primer envío. Se muestra 0..* como cardinalidad estructural; la revisión no se edita.

08. Clases: misiones y recompensas

Referencias cruzadas explícitas por identificador; las recompensas son movimientos persistidos.



Dos FK opcionales y únicas vinculan misión-actividad. Recompensa individual: máximo una por actividad/integrante.

09. Interpretación y clases complementarias

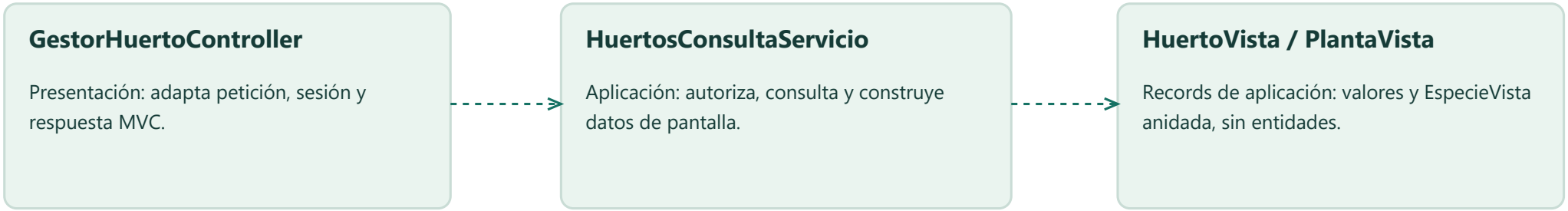
Las multiplicidades expresan estructura; las restricciones de estado completan el modelo.

Relación o clase	Interpretación y responsabilidad
Comunidad - Huerto	Cada huerto pertenece a una comunidad; comunidad_id es único en huerto. Una comunidad puede existir antes de registrar su huerto.
Integrante / ActividadParticipante	Integrante representa una membresía. ActividadParticipante resuelve la participación en actividades y conserva comunidadId para consistencia.
Especie - Planta	Especie conserva el catálogo compartido. Planta pertenece al huerto y aporta cantidad, activa y plantada. Los nombres de catálogo se resuelven al construir las vistas.
EspacioHuerto / HuertoCerramiento	Distribución física y cerramientos del huerto. EspacioHuerto registra huertoId, diseño y área. No se dibuja una asignación Planta-Espacio que el modelo no persiste.
ArticuloBodega	id y huertoId: Long; nombre: String; categoria: CategoriaArticulo; cantidad: int. actualizar(nombre, categoria, cantidad) no recibe formularios.
AsignacionRevisor / SolicitudComunidad	La primera relaciona revisor y comunidad. La segunda conserva el expediente y decisión de alta o eliminación; no es una subclase de Comunidad.

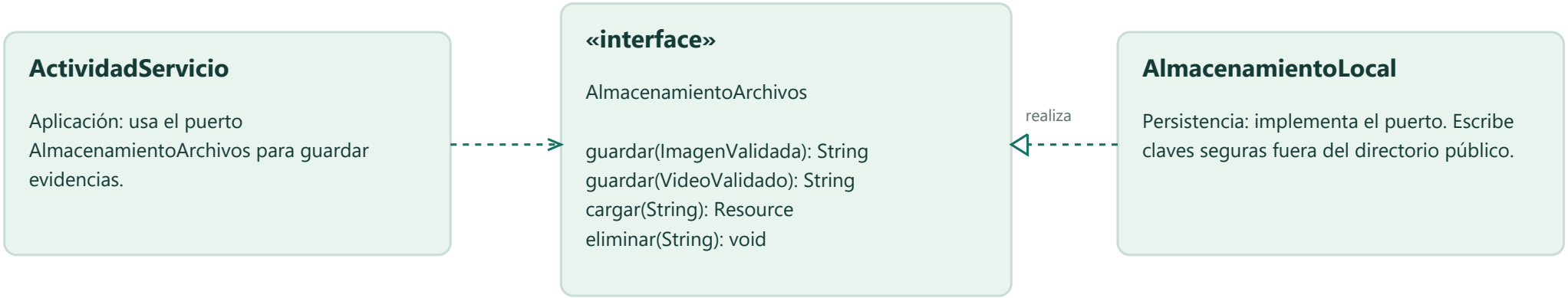
Las láminas son vistas parciales del modelo. No se emplea composición UML porque la eliminación en cascada no es su contrato.

10. Colaboración técnica de las clases

El controlador no entrega entidades al HTML: la frontera de lectura utiliza records de aplicación.



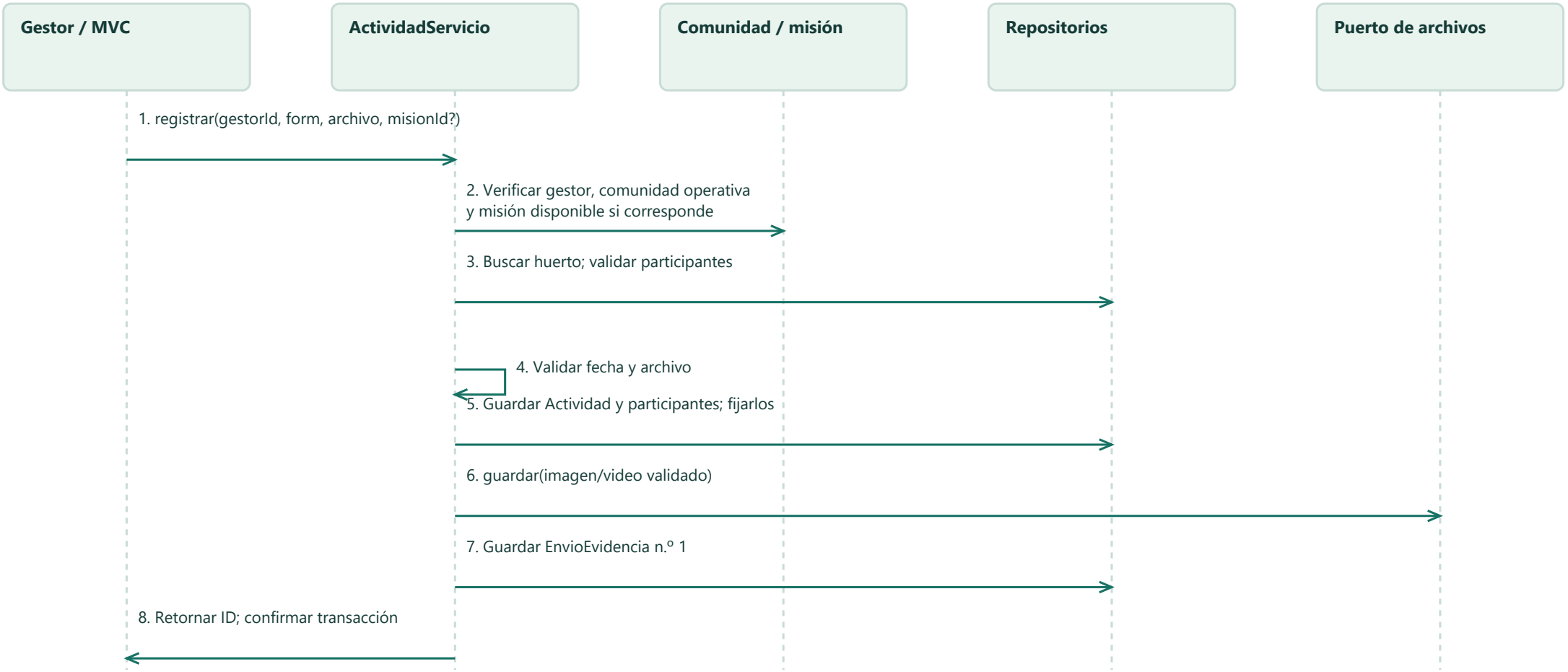
La fila superior resume una dependencia y su resultado. La inferior muestra el puerto existente para archivos. Spring Resource sigue formando parte de ese contrato; no se afirma independencia total del framework.



Lecturas dentro de la transacción + open-in-view: false. No se delegan consultas diferidas a las plantillas.

11. Secuencia: registrar actividad

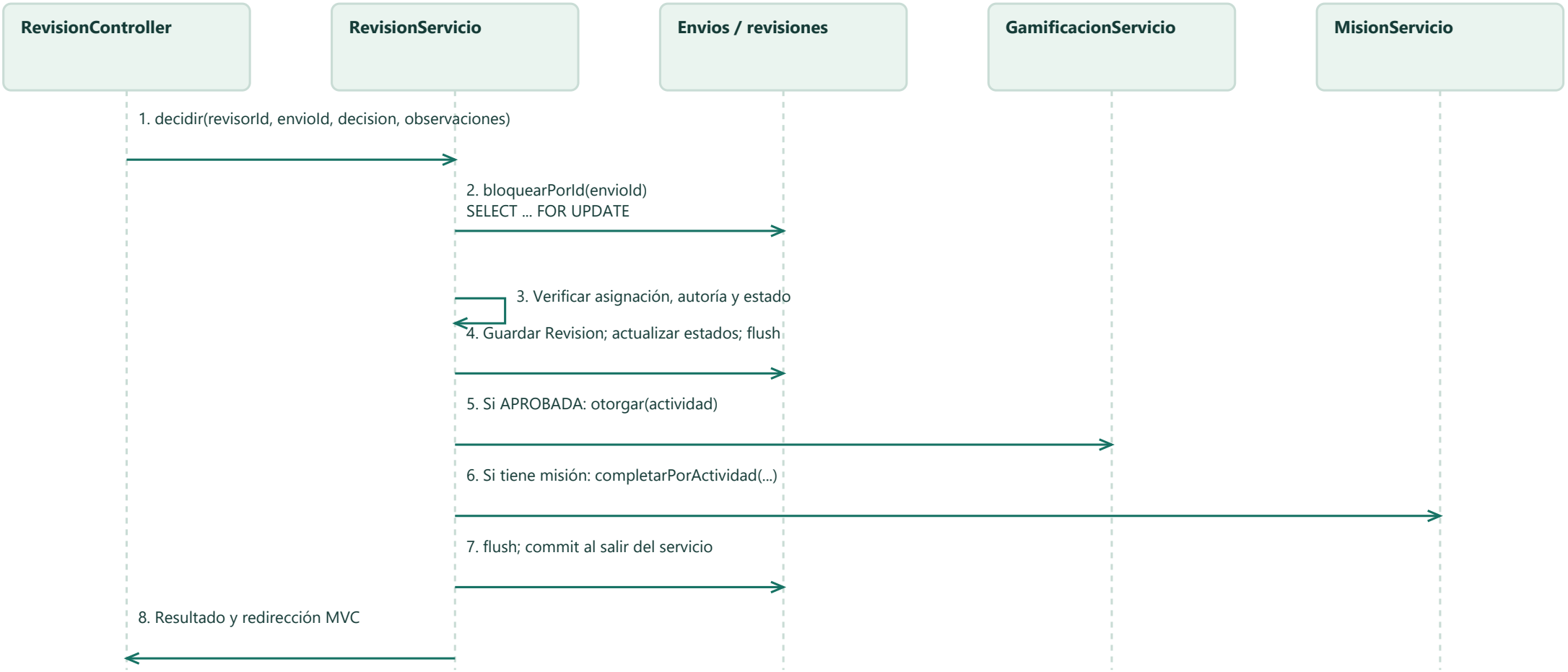
Flujo síncrono resumido; el servicio delimita la transacción de escritura.



Consistencia: el vínculo con la misión se registra si fue seleccionada. Una subida fallida cancela la operación; si la transacción revierte después de guardar el archivo, se programa su compensación. Disco y base no forman una transacción distribuida.

12. Secuencia: decidir una evidencia

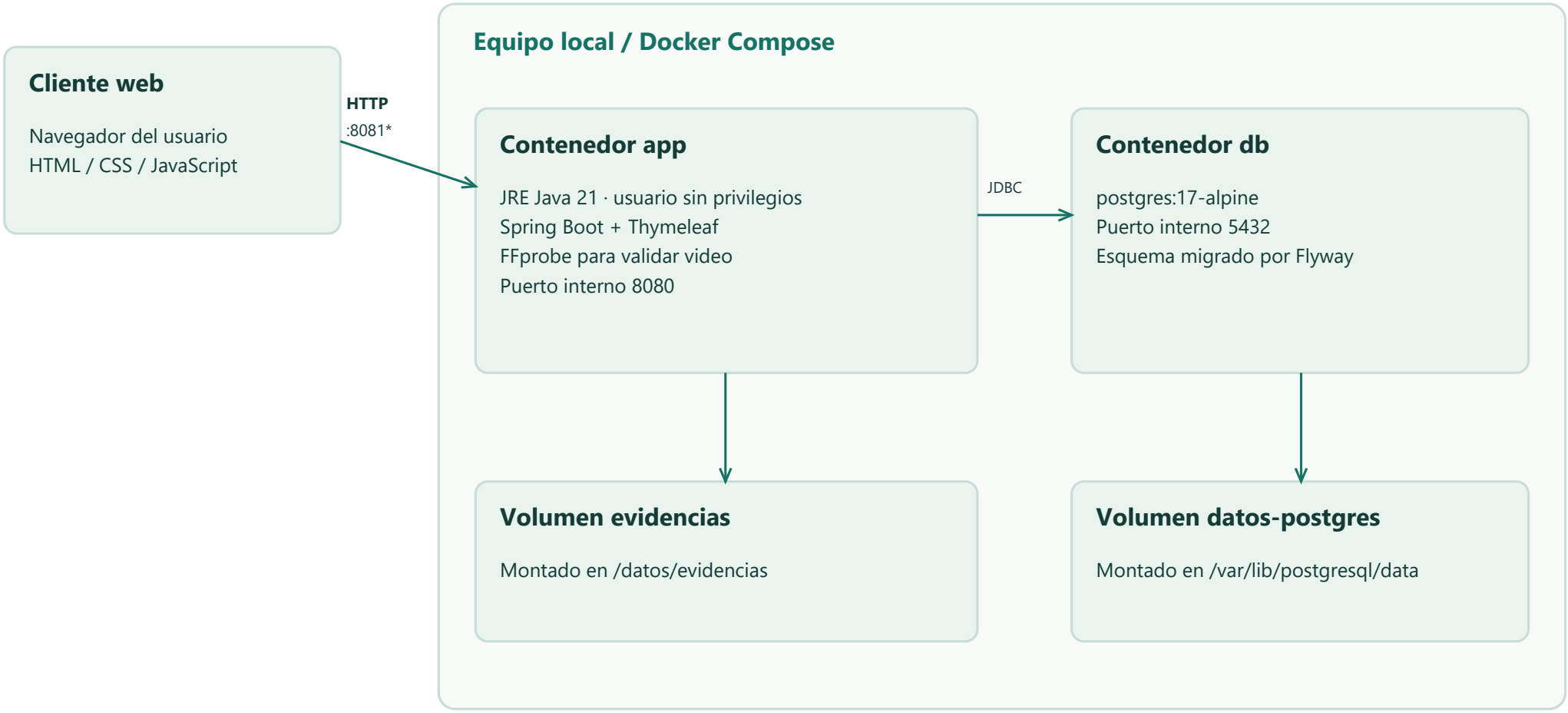
El servicio valida rol y decisión antes del bloqueo; POST /revision/envios/{envioId}/decision.



Alternativas: un envío decidido produce conflicto; rechazar exige observaciones y no genera recompensas. El reenvío crea otra evidencia y conserva el historial. El bloqueo protege la revisión del envío; no resuelve el pendiente de misiones distintas aprobadas a la vez.

13. Diagrama de despliegue

Escenario definido en compose.yaml y Dockerfile. Es documentación de configuración, no un despliegue nuevo.



*** Puertos predeterminados:** host 8081 → app 8080; base expuesta solo en 127.0.0.1:5432. El archivo .env aporta la configuración y las credenciales. La aplicación espera el estado saludable de PostgreSQL.

14. Seguridad, persistencia y operación

Controles existentes y consecuencias del diseño.

Seguridad en tres niveles

Spring Security aplica rutas, sesión y CSRF. Los servicios verifican usuario activo, rol, membresía y propiedad/asignación del recurso. PostgreSQL respalda las reglas con claves, índices y triggers. El rol ADMIN_MINISTERIO_SALUD pertenece al prototipo; no implica integración externa con un ministerio.

Esquema y transacciones

Flyway versiona la base y Hibernate valida el esquema con ddl-auto: validate. Las entidades se consultan dentro de los servicios; open-in-view está desactivado. Las transacciones de escritura conservan bloqueos e idempotencia para revisión y recompensas.

Privacidad de archivos

Fotos y videos no se sirven como archivos estáticos públicos. El acceso pasa por autorización y el adaptador valida claves/rutas. La eliminación utiliza una cola persistida con reintentos. La recuperación ante fallos requiere considerar juntos base y archivos; no se ha verificado aquí un procedimiento de restauración.

Despliegue externo descrito

render.yaml declara una aplicación Docker y PostgreSQL. No configura un disco persistente de evidencias. Por tanto, la conservación de imágenes y videos no queda resuelta en ese archivo. Este informe no afirma que se haya desplegado en Render ni verifica las condiciones comerciales del proveedor.

15. Calidad de pruebas y trazabilidad

Resultados registrados el 12/11/2026; no se han vuelto a ejecutar durante la elaboración del informe.

Fase / comprobación	Resultado y propósito
Surefire: mvn test	66 casos rápidos: unitarias, slice web y arquitectura. Sin fallos, errores ni omitidos.
Failsafe: mvn verify	111 casos de integración; PostgreSQL 17.11 externo desechable. Verificación completa: 177 casos y BUILD SUCCESS.
Repetición de integración	111 casos, segunda semilla de orden y misma base temporal. Cero fallos; no es una prueba adicional contada dentro de los 177.
Arquitectura productiva	8 reglas: fronteras de capas, SQL, entidades por ID, records, transacciones y recursos únicos.
Organización de la suite	3 reglas: paquetes y rutas coherentes, etiquetas/sufijos Test-IT y unitarias sin contexto completo Spring.
Before / After	Preparación nueva por caso, limpieza de datos y archivos fuera de la transacción, cierre de validadores y ejecutores.

TDD observado: dos contratos de organización se ejecutaron primero en rojo y luego pasaron tras refactorizar. No se atribuye TDD histórico al negocio. No se midió cobertura ni mutación; el resultado verde se limita a los escenarios declarados.

16. Decisiones y límites de la versión

La conformidad de capas no equivale a ausencia de defectos funcionales.

Decisión conservada	Consecuencia de diseño
IDs en lugar de relaciones JPA	Joins y proyecciones explícitos; sin cascadas implícitas entre entidades. Aplicación debe resolver datos relacionados.
Servicios + DTO de lectura	La interfaz no accede a repositorios ni recibe entidades. Los permisos y la materialización de vistas se centralizan.
Monolito con transacción compartida	Revisión y recompensas pueden confirmarse juntas. Crece el acoplamiento de coordinación entre módulos.
Archivo local con puerto	Puede reemplazarse el adaptador manteniendo su contrato. La durabilidad depende del volumen y la estrategia de respaldo.

Pendientes funcionales documentados

Persisten casos de eliminación con relaciones nuevas; desbloqueo de misiones bajo aprobaciones simultáneas; sincronización de cerramientos; cupo al cambiar el tipo de comunidad; reparto de áreas pequeñas y estimación del ranking con recompensas variables. Requieren corrección y regresiones específicas antes de afirmar que están resueltos.

Las recompensas y umbrales de demostración son provisionales. Enumerar recurrencia no demuestra ejecución recurrente de misiones.

17. Fuentes y correspondencia de entrega

Fuentes primarias de la copia local, revisadas para este documento.

Fuente dentro del repositorio	Uso en el informe
docs/arquitectura.md	Estilo, módulos, dependencias permitidas, entidades por ID, DTO y límites arquitectónicos.
src/main/java/ec/edu/epn/huertos/	Clases de dominio, firmas de métodos, servicios y controladores. Láminas UML y secuencias contrastadas con estas fuentes.
src/main/resources/db/migration/	Restricciones y esquema. Junto con anotaciones JPA, respalda la lectura de cardinalidades.
pom.xml; frontend/; application.yml	Java 21, Spring Boot 4.1.2, empaquetado de interfaz, persistencia y configuración de archivos.
compose.yaml; Dockerfile; render.yaml	Topología de despliegue, puertos, volúmenes y límites del almacenamiento externo descrito.
docs/calidad-pruebas.md; docs/verificacion-pruebas.txt	Evidencia de las 177 pruebas, repetición e infraestructura efectivamente utilizada.
docs/cambios-arquitectura.md	Alcance de la refactorización y defectos funcionales pendientes.

Correspondencia con la actividad: arquitectura del software: secciones 02, 03, 05 y 14; diagrama de arquitectura: 04; diagrama de clases: 06-10; secuencias: 11-12; despliegue: 13. Las láminas de clases se entregan también juntas en PDF y separadas en SVG para los aportes al documento 03_Diseño_V1.